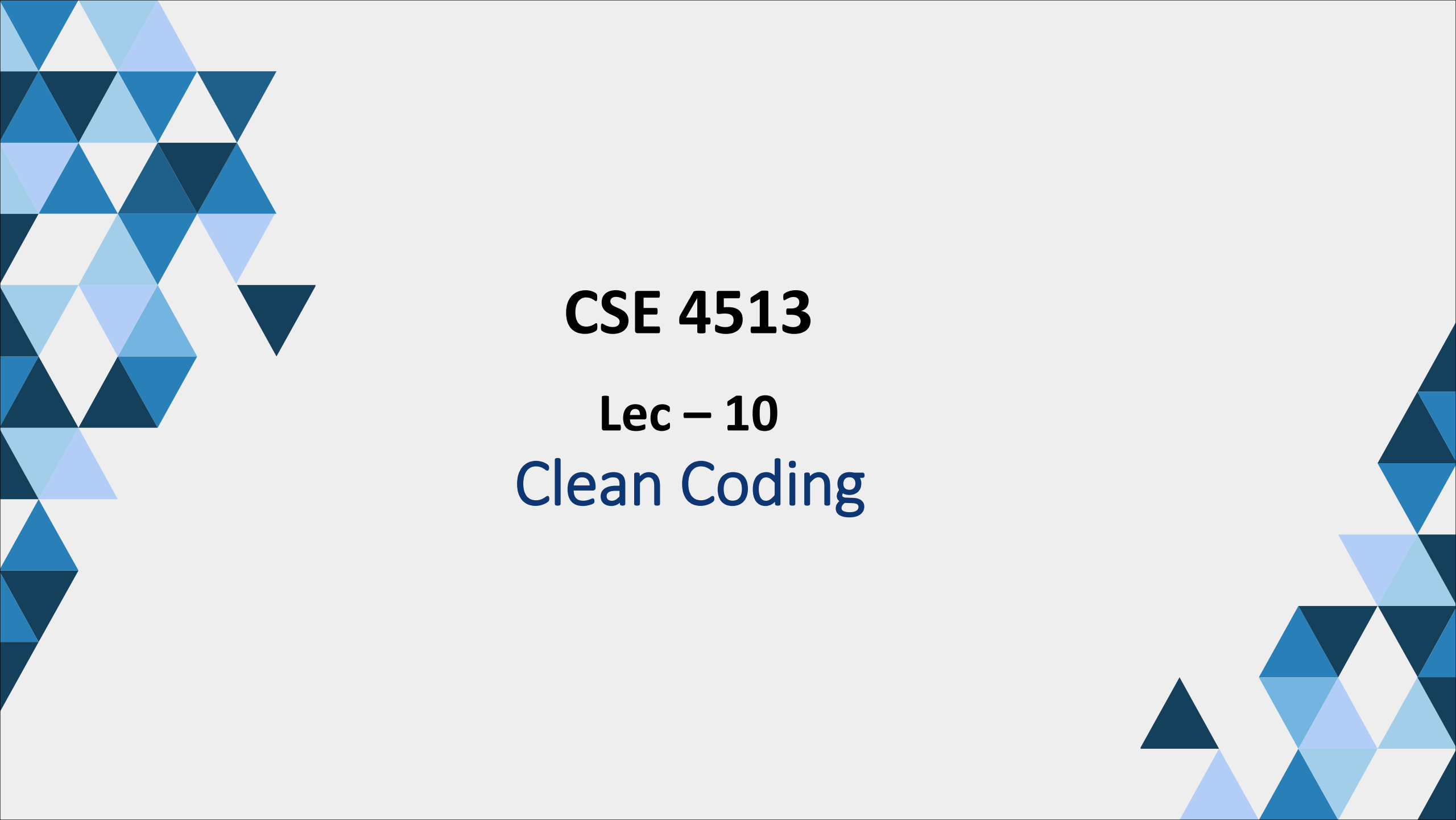




# **CSE 4513**

## **Lec – 10**

### **Clean Coding**

# CODING



Developers .....by definition they code.

For some of them it's just a job, for others it is a hobby, a craft or even an art !

You need concentration and focus to achieve the 4 main goals of coding :

1. Your code must work !

2. Your code must solve the problem !

3. Your code must fit well into the existing system !  
Any tool can write code that a computer can understand.

4. Your code must be readable by other programmers ! (Clean

Code)

~Martin Fowler

# CLEAN CODE – WHAT'S THAT



**T**estable and tested

**E**legant

**E**fficient

**E**xpressive

**R**eadable

**M**inimal Dependencies

**M**aintainable

**S**elf Documenting

**S**imple

**Michael Feathers, author of  
*Working Effectively with Legacy  
Code***

*I could list all of the qualities that I notice in clean code, but there is one overarching quality that leads to all of them. Clean code always looks like it was **written by someone who cares**. There is nothing obvious that you can do to make it better. All of those things were thought about by the code's author, and if you try to imagine improvements, you're led back to where you are, sitting in appreciation of the code someone left for you—code left by someone who cares deeply about the craft.*



**Bjarne Stroustrup, inventor of C++**

*I like my code to be elegant and efficient. The logic should be straightforward to make it hard for bugs to hide, the dependencies minimal to ease maintenance, error handling complete according to an articulated strategy, and performance close to optimal so as not to tempt people to make the code messy with*



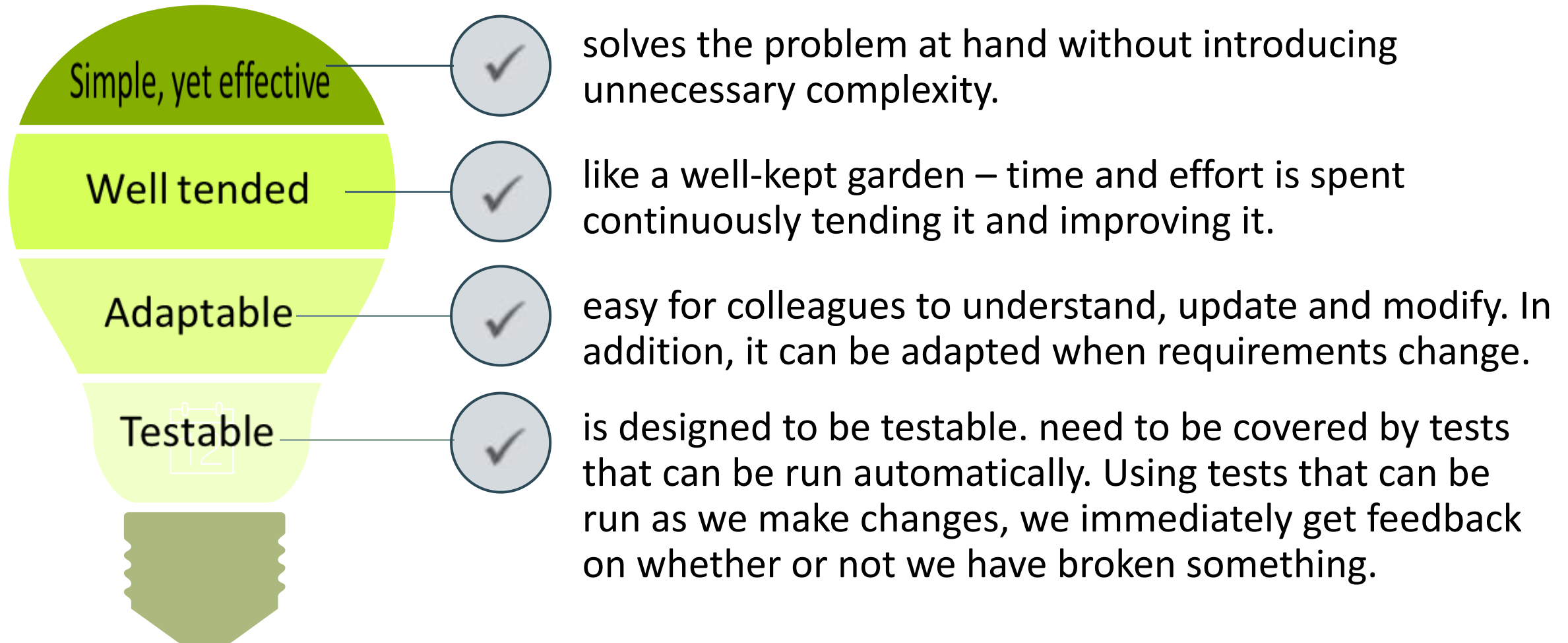
**Grady Boock, author of *Object Oriented Analysis and Design with Applications***

*Clean code is simple and direct. Clean code reads like well-written prose. Clean code never obscures the designer's intent but rather is full of crisp abstractions and straightforward lines of control.*

# WHAT EXACTLY IS CLEAN CODE?



”software code that is formatted correctly and in an organized manner so that another coder can easily read or modify it”.



# BAD CODE – WHAT'S THAT

---

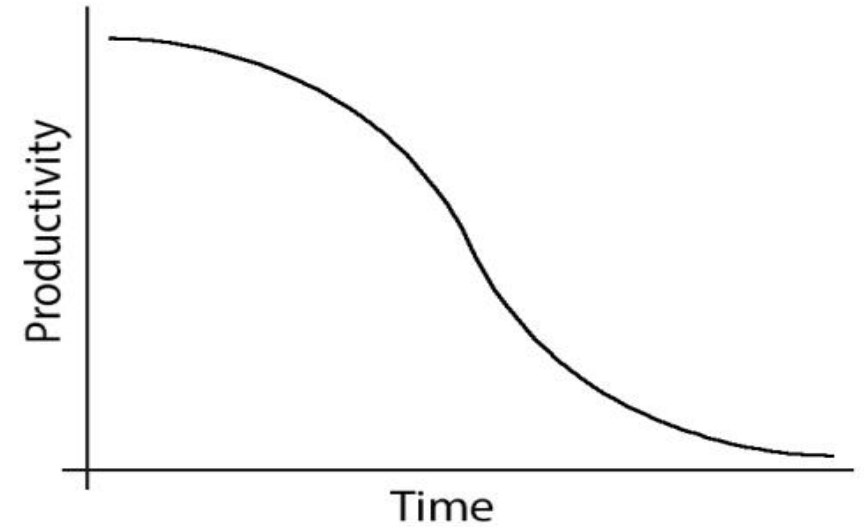
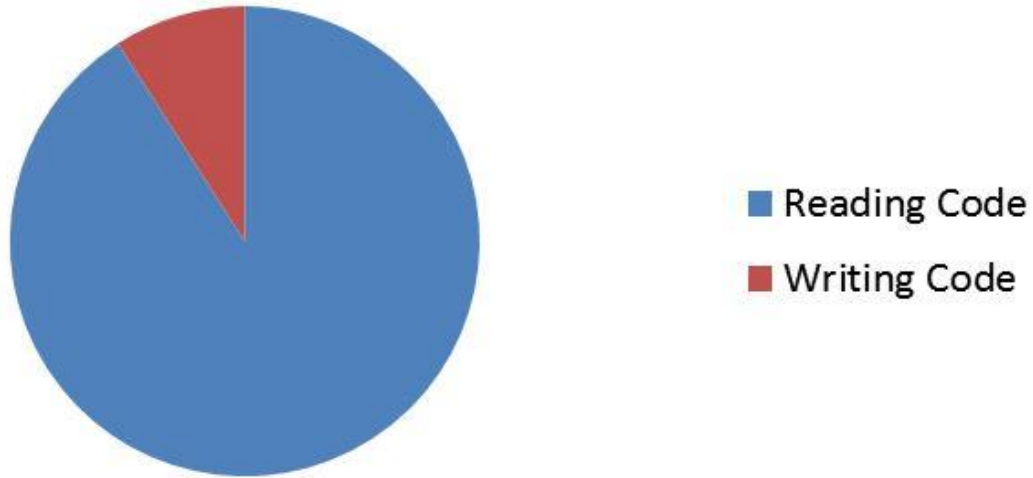


- Inappropriate Information
- Obsolete Comments/ Redundant Comments / Poorly Written Comments
- Commented out code
- Too many arguments
- Dead functions
- Duplication
- Incorrect Behavior and Boundaries
- Code at wrong level of abstraction
- Base classes depending on their derivatives
- Too much information
- ....

# WHY IS CLEAN CODE IMPORTANT



Time spent by programmers at work



- Clean Code saves time
- Maintains fast productivity
- To make next developer Happier

# WHY IS CLEAN CODE IMPORTANT



*Every time you code like this you make a programme*

# WHAT PREVENTS CLEAN CODE

---



- Ignorance
- Stubbornness
- Short-Timer Syndrome
- Arrogance
- Job Security



# WHAT PREVENTS CLEAN CODE

---



Number one reason:

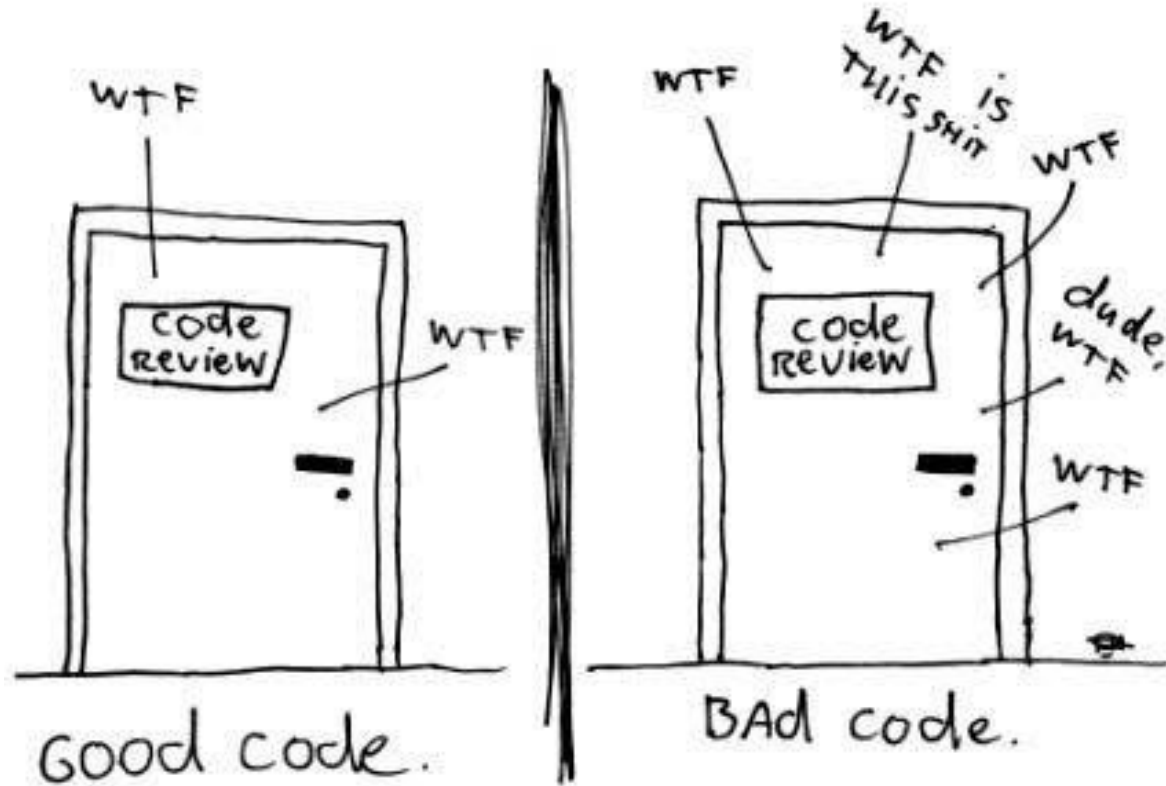
“I’ll clean it up later.”

Pro Tip: “Later” never comes.

# MEASURING CODE QUALITY

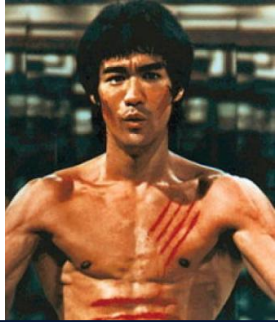


The ONLY valid measurement  
of code quality: WTFs/minute



(c) 2008 Focus Shift

# WRITING CLEAN CODE



Jeet Kune Do



Drunken Master?



Wing Chun



Code Complete



S.O.L.I.D



TDD

**JUST LIKE MARTIAL ARTS HAS MANY MASTERS WITH DIFFERENT STYLES, SO DOES CLEAN CODE**

# WHAT MOTIVATE US TO WRITE CLEAN CODE

---



When Robert Baden-Powell, the founder of the Scout Movement, passed away in 1941 a famous letter was found in his drawer. It was addressed to the scouts of the world and contained a line which is now classic:

**“TRY AND LEAVE THIS WORLD A LITTLE BETTER THAN YOU FOUND IT.”**

The Baden-Powell appeal can also serve as a guideline for developers –

**”LEAVE THE CODE CLEANER THAN YOU FOUND IT!”**

# WHAT MOTIVATE US TO WRITE CLEAN CODE



Another Rule of Thumb

**“IMAGINE THAT THE DEVELOPER WHO  
COMES AFTER YOU IS A VIOLENT  
PSYCHOPATH WHO KNOWS WHERE YOU  
LIVE.”**



# BUT HOW DO WE DO IT???

---



- Naming
- Functions
- Comments rules
- Source code structure
- Objects and data structures
- Error Handling
- Tests
- Code smells

# NAMING: USE INTENTION-REVEALING NAMES



*Bad*

```
int d; // elapsed time in days
```

*Good*

```
int elapsedTimeInDays;  
int daysSinceCreation;  
int daysSinceModification;  
int fileAgeInDays;
```



Choosing good names takes time, but saves more than it takes.  
So take care with your names and change them when you  
find better ones.

If a name requires a comment, then the name does not  
reveals its intent.



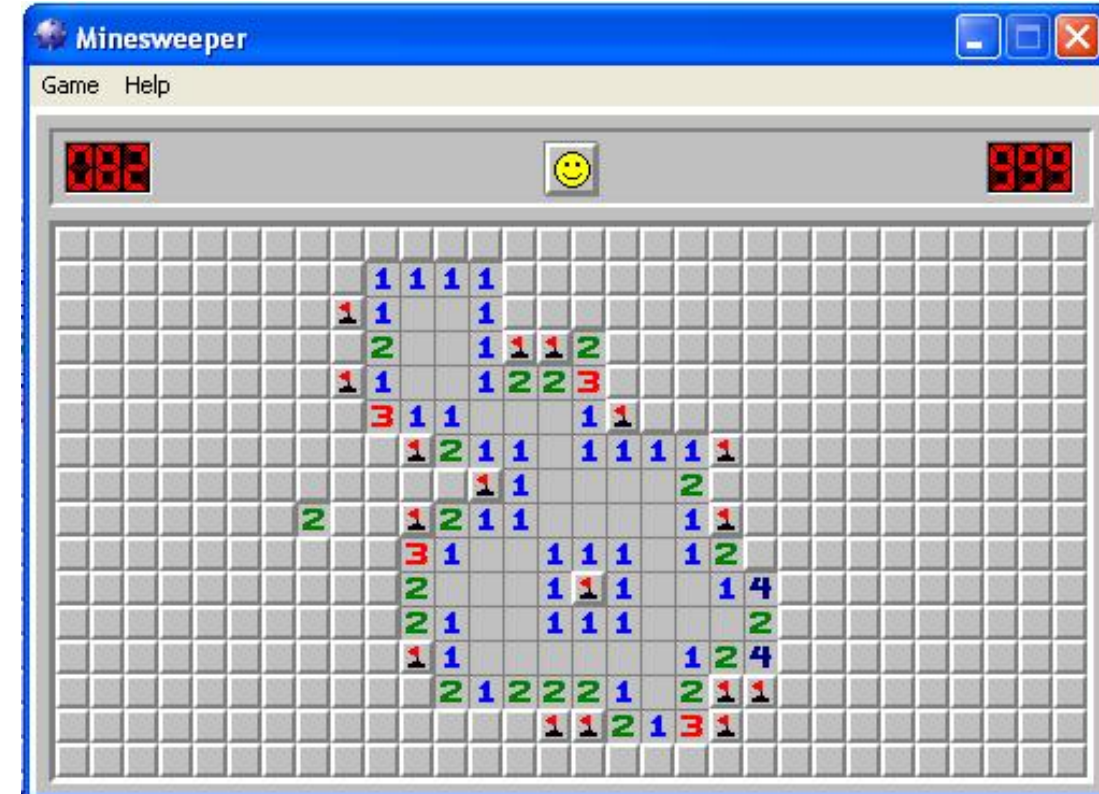
# NAMING : USE INTENTION-REVEALING NAMES



*Bad*

```
public List<int[]> getThem() {  
    List<int[]> list1 = new  
    ArrayList<int[]>();  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```

1. What is the content of the List1?
2. What is the significance of the item x[0] in the list?.
3. Why we compare x[0] vs 4?
4. How would i use the returned list?





# NAMING : USE INTENTION-REVEALING NAMES



**Bad**

```
public List<int[]> getThem() {  
    List<int[]> list1 = new ArrayList<int[]>();  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```

**Good**

```
public final static int STATUS_VALUE = 0;  
public final static int FLAGGED = 4;  
  
public List<int[]> getFlaggedCells() {  
    List<int[]> flaggedCells = new ArrayList<int[]>();  
    for (int[] cell : gameBoard)  
        if (cell[STATUS_VALUE] == FLAGGED)  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```

What we have done:

1. Use intention revealing names  
flaggedCells  
rather than list1
2. Replaced magic numbers with constants  
cell[STATUS\_VALUE] rather than x[0]

# NAMING : AVOID DISINFORMATION



- Must avoid leaving false clues that obscure the meaning of code.
- Should avoid words whose entrenched meaning vary from our intended meaning.

Don't refer to a group of accounts as "accountList," whereas it is actually not a List. Instead, name it "accountGroup," "accounts," or "bunchOfAccounts."

Avoid unnecessary encodings of datatypes along with the variable name.varies

```
String nameString;
```

```
Float salaryFloat;
```

# NAMING : MAKE MEANINGFUL DISTINCTION



- If there are two different things in the same scope, you might be tempted to change one name in an arbitrary way.

```
String cust;
```

What is stored in cust, which is different from customer?

```
String customer;
```

```
class ProductInfo
```

How are these classes different?

```
class ProductData
```

```
void getActiveAccount ()
```

```
void getActiveAccounts ()
```

How do you differentiate between these methods in the same class?

```
void getActiveAccountsInfo ()
```

suggestion here is to make meaningful distinctions.

# NAMING : MAKE MEANINGFUL DISTINCTION



- the arguments are non-informative, a1 and a2 doesn't provide clues to the author intention.

```
public static void copyChars(char a1[], char a2[]) {  
    for (int i = 0; i < a1.length; i++) {  
        a2[i] = a1[i];  
    }  
}
```



```
public static void copyChars(char source[], char destination[]) {  
    for (int i = 0; i < source.length; i++) {  
        destination[i] = source[i];  
    }  
}
```

# NAMING : USE PRONOUNCEABLE NAMES



Ensure that names are pronounceable. Nobody wants a tongue twister.

*Bad* **class** DtaRcrd102 {  
    **private** Date **genymdhms**;  
    **private** Date **modymdhms**;  
    **private final** String **pszqint** = "102";  
};

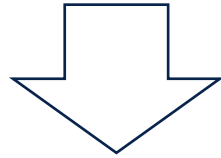
*Good* **class** Customer {  
    **private** Date **generationTimestamp**;  
    **private** Date **modificationTimestamp**;  
    **private final** String **recordId** = "102";  
};

# NAMING : USE SEARCHABLE NAMES



- Avoid single letter names and numeric constants, if they are not easy to locate across the body of the text.

```
for (int j = 0; j < 34; j++) {  
    s += (t[j] * 4) / 5;  
}
```



```
int realDaysPerIdealDay = 4;  
const int WORK_DAYS_PER_WEEK = 5;  
int sum = 0;  
for (int = 0; j < NUMBER_OF_TASKS; j++) {  
    int realTaskDays = taskEstimate[j] *  
    realDaysPerIdealDay;  
    int realTaskWeeks = (realdays /  
    WORK_DAYS_PER_WEEK);  
    sum += realTaskWeeks;  
}
```

*Sum here is not really useful but at least is searchable. It will be easier to find **WORKS\_DAYS\_PER\_WEEK** than to find all the places where 5 was used.*



# NAMING : DON'T BE CUTE/DON'T USE OFFENSIVE WORDS

---

If names are too clever, they will be memorable only to people who share your sense of humor, and only as long as these people remember the joke.

Will they know what the function named **HolyHandGrenade** is supposed to do?

Sure, it's cute, but maybe in this case **DeleteItems** might be a better name.

Avoid using "**HastaLaVistaBaby()**" to mean "**terminateProcess()**" or "**endExecution()**". This is a famous quote from the Terminator movie series but can be confusing.

**Say what you mean. Mean what you say.**

# NAMING : PICK ONE WORD PER CONCEPT

---



Use the same concept across the codebase.

## **FetchValue() vs GetValue() vs RetrieveValue()**

If you are using **fetchValue()** to return a value of something, use the same concept; instead of using **fetchValue()** in all the code, using **getValue()**, **retrieveValue()** will confuse the reader.

## **dataFetcher() vs dataGetter() vs dataRetriver()**

If all three methods do the same thing, don't mix and match across the code base. Instead, stick to one.



# NAMING : DON'T JUST JOKE

---



- This is exactly opposite of previous rule.
- Avoid using the same word for two different purposes.
- Using the same term for two different ideas is essentially pun.

For example, say you have two classes having the same `add()` method. In one class, it performs the addition of two values, in another, it inserts an element into a list. So, choose wisely; in the second class use `insert()` or `append()` instead of `add()` . .

# NAMING : ADD MEANINGFUL CONTEXT



- Most names are not meaningful.
- you have to place names in context for your reader by enclosing them in well-named classes, functions, or namespaces.
- If not possible, prefixing the names may be necessary as a last resort.

For example, the member variable named "state" inside a address class shows what it contains. But the same "state" may be misleading if used to store "Name of The State" without context. So, in this scenario, name it something like "addressState" or "StateName."

```
public class Address {  
    private String addressState; // Descriptive variable name  
}
```

# MORE ON NAMING

---



## ✓ Class Names:

- Class names should be in CamelCase, starting with an uppercase letter.
- Use nouns or noun phrases for class names (e.g., Car, EmployeeRecord, AccountManager).

## ✓ Interface Names:

- Interface names should be in CamelCase, starting with an uppercase letter.
- Use adjectives or adjective phrases for interface names (e.g., Serializable, runnable).

## ✓ Method Names:

- Method names should be in camelCase, starting with a lowercase letter.
- Use verbs or verb phrases for method names (e.g., calculateTotal(), printReport()).

# MORE ON NAMING

---



## ✓ Constant Names:

- Constant names should be in uppercase with underscores separating words (e.g., MAX\_VALUE, PI).
- Use all uppercase letters for constant names.

## ✓ Method Parameters:

- Use meaningful and descriptive names for method parameters.
- Avoid single-letter parameter names like "a," "b," or "c."

## ✓ Getter and Setter Methods:

- Use the name of the property in the method name for getter and setter methods (e.g., getFirstName(), setLastName()).

# NAMING : SUMMARY

---



1. Meaningful names
2. Avoid Disinformation
3. Avoid small variations in names – s1, s2
4. Meaningful Distinctions
  - `if (!fundsAvailable || availableFunds < -0.004m)`
5. Pronounceable Names - rptparam
6. Searchable Names – MAX\_DAYS instead of 999
7. Avoid Encodings – e.g. strName, strFName, Compiler
8. Function Names are verbs
9. Data fields are nouns



*“Don’t comment bad code—rewrite it.”*  
—Brian W. Kernighan and P. J. Plauger

The best kind of comments are the ones you don't need

# COMMENTS



- Rule #1: Comments lie
  - ✓ Code is updated or moved, but not the comments.



# COMMENTS



- Rule #2: Comments do not make up for bad code
  - ✓ If the code is that unclear, rewrite the code
- Rule #3: Explain yourself in Code

```
// Check to see if the employee is eligible for full benefits  
if ((employee.flags & HOURLY_FLAG) && (employee.age > 65))
```

*Should be*

```
if (employee.isEligibleForFullBenefits())
```



# GOOD COMMENTS

---



## ➤ Legal Comments

- ✓ We should write some legal comment like this to help another person know where and when the code come from

```
// Copyright (C) 2003,2004,2005 by Object Mentor, Inc. All  
rights reserved. // Released under the terms of the GNU  
General Public License version 2 or later.
```

# GOOD COMMENTS



## ➤ Explanation of Intent

```
assertTrue(a.compareTo(a) == 0); // a == a
```

```
assertTrue(a.compareTo(b) != 0); // a != b
```

```
assertTrue(ab.compareTo(ab) == 0); // ab == ab
```

## ➤ Warning of Consequence

```
public static SimpleDateFormat makeStandardHttpDateFormat()  
{  
    //SimpleDateFormat is not thread safe,  
    //so we need to create each instance independently.  
    SimpleDateFormat df = new SimpleDateFormat("EEE, dd MMM yyyy  
    HH:mm:ss z");  
    df.setTimeZone(TimeZone.getTimeZone("GMT"));  
    return df;  
}
```

# GOOD COMMENTS



## ➤ TODO comments

- ✓ We should write TODO comment for note something that we will do in function

```
// TODO: Add password hashing for better security
public String hashPassword(String password) {
    // Incomplete code for password hashing
    // TODO: Implement a secure password hashing algorithm
    // This is just a placeholder and should be replaced
    return password;
}
```

# GOOD COMMENTS



## ➤ Amplification

- ✓ A comment may be used to amplify the importance of something that may otherwise seem insignificant.

```
public class OrderProcessingService {  
    public boolean processOrder(Order order) {  
        // Check if the customer has sufficient funds to place the order  
        if (order.getTotalAmount() >  
order.getCustomer().getAccountBalance()) {  
            // The customer's account balance is insufficient to place the order.  
            // Consider implementing a notification or payment processing logic.  
            return false;  
        }  
        // Continue with order processing...  
        return true;  
    }  
}
```

# BAD COMMENTS



## ➤ Mumbling

- ✓ Placing in a comment just because you feel you should or because the process requires it, is a hack.
- ✓ If you decide to write a comment, then spend the time necessary to make sure it is the best comment you can write. Example:

```
// This function calculates the sum of two numbers and returns the result  
public int calculateSum(int a, int b) {  
    return a + b;  
}
```

*What is the necessity of this comment?*

# BAD COMMENTS



## ➤ Redundant Comments

// The method orchestrates the creation of an Employee by combining a person's name and a unique identifier to establish their identity.

```
public Employee(String name, int employeeID) {  
    this.name = name;  
    this.employeeID = employeeID;  
}
```

*What purpose does the comment serve?  
It's certainly not more informative than the code.  
It is not easier to read than the code.*

# BAD COMMENTS



## ➤ Mandated Comments

- ✓ It is just plain silly to have a rule that says that every function must have a javadoc, or every variable must have a comment. Comments like this just mess up the code, propagate lies, and lend to general confusion and disorganization.

```
function hashIt(data) {  
    // The hash  
    let hash = 0;  
  
    // Length of string  
    const length = data.length;  
  
    // Loop through every character in data  
    for (let i = 0; i < length; i++) {  
        // Get character code.  
        const char = data.charCodeAt(i);  
        // Make the hash  
        hash = (hash << 5) - hash + char;  
        // Convert to 32-bit integer  
        hash &= hash;  
    }  
}
```



# BAD COMMENTS



## ➤ Journal Comments

- ✓ Remember, use version control! There's no need for dead code, commented code, and especially journal comments. Use git log to get history!

```
/**  
 * 2016-12-20: Removed monads, didn't understand them (RM)  
 * 2016-10-01: Improved using special monads (JP)  
 * 2016-02-03: Removed type-checking (LI)  
 * 2015-03-14: Added combine with type-checking (JR)  
 */  
function combine(a, b) {  
    return a + b;  
}
```

# BAD COMMENTS



## ➤ Noise Comments.

```
/**  
 * Default constructor.  
 */  
protected AnnualDateRule() {  
}  
/** The day of the month. */  
private int dayOfMonth;
```

*The comments in the above examples doesn't provides new information*

## ➤ Position Markers

```
// Actions //////////////////////////////////////
```

*This type of comments are noising*

# BAD COMMENTS



➤ Don't Use a Comment When You Can Use a Function or a Variable

*// Calculate the total cost of items in the shopping cart*

```
double totalCost = 0;
for (Item item : shoppingCart) {
    totalCost += item.getPrice();
}
```

*Can be modified using a function and a variable*

```
double totalCost = calculateTotalCost(shoppingCart);
```

```
double calculateTotalCost(List<Item> shoppingCart) {
    double totalCost = 0;
    for (Item item : shoppingCart) {
        totalCost += item.getPrice();
    }
    return totalCost;
}
```

# BAD COMMENTS



## ➤ Closing Brace Comments

```
try {
    while ((line = in.readLine()) != null) {
        lineCount++;
        charCount += line.length();
        String words[] = line.split("\\W");
        wordCount += words.length;
    } //while
    System.out.println("wordCount = " + wordCount);
    System.out.println("lineCount = " + lineCount);
    System.out.println("charCount = " + charCount);
} // try
catch (IOException e) {
    System.err.println("Error:" + e.getMessage());
} //catch
```

# BAD COMMENTS



## ➤ Attributions

```
/*  
* Copyright (c) 2023 by John Doe  
* Licensed under the Apache License, Version 2.0 (the "License").  
* You may obtain a copy of the License at  
* http://www.apache.org/licenses/LICENSE-2.0  
*/
```

## ➤ Commented-Out Code

```
doStuff();  
// doOtherStuff();  
// doSomeMoreStuff();  
// doSoMuchStuff();
```

*VCS exists for a reason. Leave old code in your history.*

# COMMENT SUMMARY

---



- ✓ Always try to explain yourself in code.
- ✓ Don't be redundant.
- ✓ Don't add obvious noise.
- ✓ Don't use closing brace comments.
- ✓ Don't comment out code. Just remove.
- ✓ Use as explanation of intent.
- ✓ Use as clarification of code.
- ✓ Use as warning of consequences.

# FORMATTING



```
function register()
{
    if (!empty($_POST)) {
        $msg = '';
        if ($_POST['user_name']) {
            if ($_POST['user_password_new']) {
                if ($_POST['user_password_new'] === $_POST['user_password_repeat']) {
                    if (strlen($_POST['user_password_new']) > 5) {
                        if (strlen($_POST['user_name']) < 65 && strlen($_POST['user_name']) > 1) {
                            if (preg_match('/^[a-z\d]{2,64}$/i', $_POST['user_name'])) {
                                $user = read_user($_POST['user_name']);
                                if (!isset($user['user_name'])) {
                                    if ($_POST['user_email']) {
                                        if (strlen($_POST['user_email']) < 65) {
                                            if (filter_var($_POST['user_email'], FILTER_VALIDATE_EMAIL)) {
                                                create_user();
                                                $_SESSION['msg'] = 'You are now registered so please login';
                                                header('Location: ' . $_SERVER['PHP_SELF']);
                                                exit();
                                            } else $msg = 'You must provide a valid email address';
                                        } else $msg = 'Email must be less than 64 characters';
                                    } else $msg = 'Email cannot be empty';
                                } else $msg = 'Username already exists';
                            } else $msg = 'Username must be only a-z, A-Z, 0-9';
                        } else $msg = 'Username must be between 2 and 64 characters';
                    } else $msg = 'Password must be at least 6 characters';
                } else $msg = 'Passwords do not match';
            } else $msg = 'Empty Password';
        } else $msg = 'Empty Username';
        $_SESSION['msg'] = $msg;
    }
    return register_form();
}
```



# FORMATTING RULES

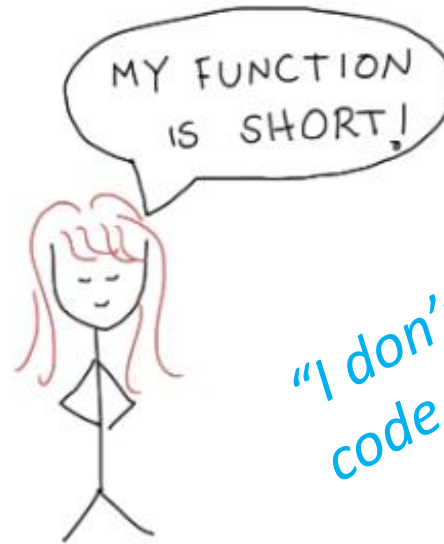
---



1. Separate concepts vertically.
2. Related code should appear vertically dense.
3. Declare variables close to their usage.
4. Dependent functions should be close.
5. Similar functions should be close.
6. Place functions in the downward direction.
7. Keep lines short.
8. Don't use horizontal alignment.
9. Don't break indentation.



# FUNCTIONS



*"I don't understand how this code works", "this code is a mess", "it's hard to modify this code"*

*If your function occupies a huge screen entirely, it doesn't count as a short function*

# FUNCTIONS



Several best practices on how to write plain, understandable functions

1. Functions should be small !!! Really
2. Block and indenting
3. Do one thing! (Single Responsibility Principle)
4. Function names should say what they do
5. Remove duplicate code
6. Avoid Side Effects
7. Remove dead code
8. One level of abstraction
9. Low number of arguments
10. Command Query Separation

# FUNCTIONS

---



## 1. Functions should be small !!! Really

- 1<sup>st</sup> rule – Function should be small.
- 2nd rule – Functions should be smaller than that!
- Functions should barely be 20/30 lines long...
- a shorter function will do less (only one simple thing).
- Additionally to this, it will be more easily to understand and manage it.

## 2. Block and indenting

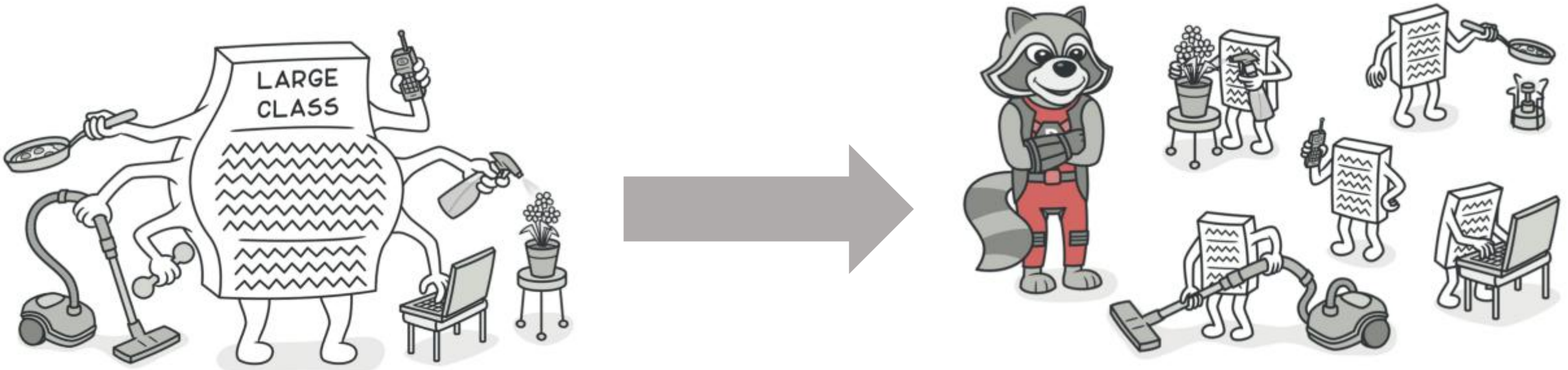
- keep the code blocks inside `if`, `else` and `while` statements minimal.
- preferably up to one line and that one line should be a function call
- Similarly, we should avoid nesting such structures inside functions by moving the nested logic into another function.
- The indent levels of a function probably shouldn't be more than two tabs!

# FUNCTIONS



## 3. Do one thing! (Single Responsibility Principle)

*“Functions should do one thing. They should do it well. They should do it only”.*

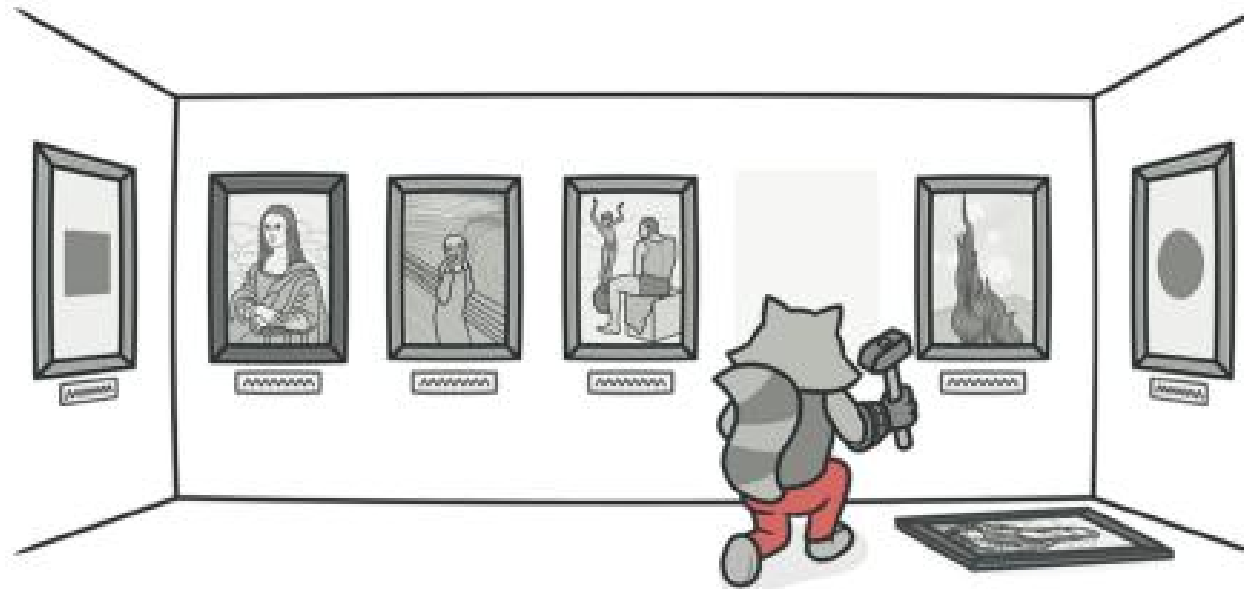


## 4. Function names should say what they do

- A function's name should explain what exactly it is doing. Nothing less, no more.
- The name of a variable, function, or class, should answer the following:
  - ✓ Why it exists?
  - ✓ What does it do?
  - ✓ How it is used?

## 5. Remove duplicate code

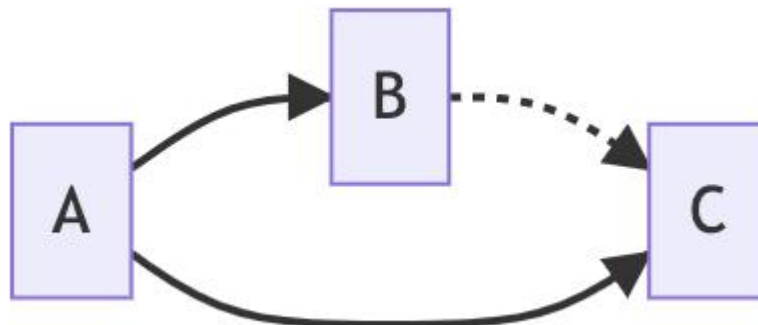
- attempt to prevent duplication of code.
- If there is duplication of code, then you may need to change many places for a single change.



## 6. Avoid Side Effects

- This is the case when your function is doing more than one thing, but without telling to the client.
- In particular our functions should not create **temporal coupling** with other function

*Temporal coupling is a kind of coupling where code is dependent on time in some*

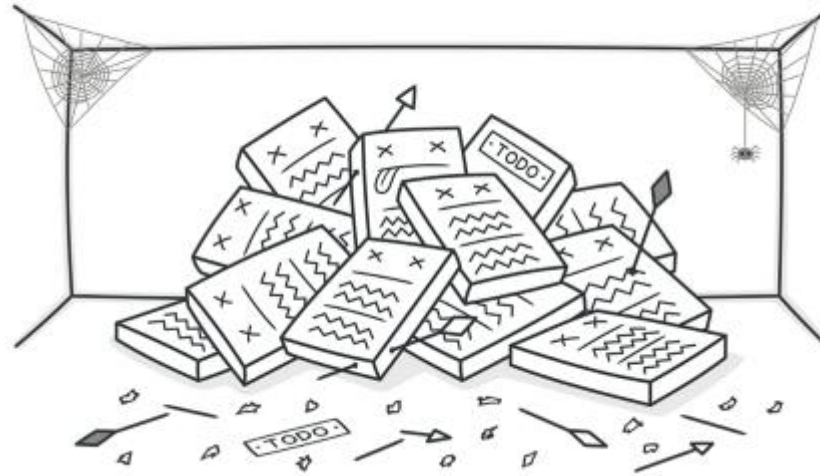


After the task of the A component is completed, B and C must be executed. In addition, B is earlier than C.



## 7. Remove dead code

- Dead code is as poor as double code.
- There's nothing in the coding base to hold it.
- If you ever need it, it will remain secure in your version history.



## 8. One level of Abstraction

- a mechanism that can tell us that the function is doing too many things.
- function must contains instruction that are at the same level of abstraction.
- For example, if a method builds and executes HTTP requests, it shouldn't also process the resulting JSON. These are different abstraction levels: one is about **communication over HTTP** and the other is about **specific data format and layout**.

## 9. Low number of arguments

- How much arguments should a perfect clean code function have?
- Zero , monadic (1 argument), dyadic (2 arguments) or triads (3 arguments) functions.
- Functions with more than 3 arguments should be avoided whenever possible.
- When the number of arguments start to growth to more than 3 arguments it is possible wrap them in specific class that describe better their meaning.

```
/// Simple version
Circle makeCircle(double x, double y, double radius);

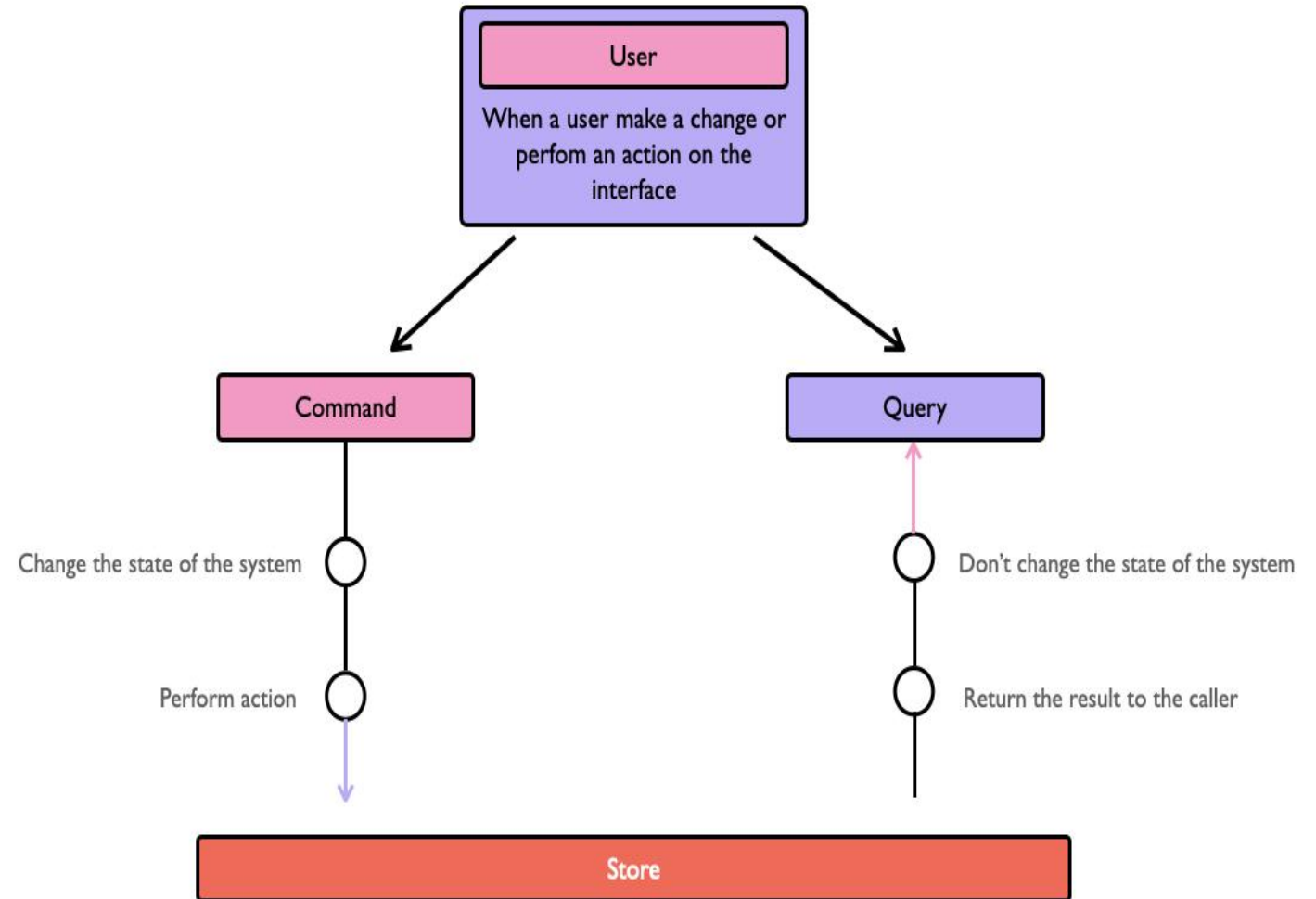
/// With arguments objects
Circle makeCircle(Point center, double radius);
```

# FUNCTIONS



## 10. Command Query Separation

- a method is either a **COMMAND** that performs an action OR a **QUERY** that returns data to the caller, but never both.
- Doing both often leads to confusion.



# FUNCTIONS



## 10. Command Query Separation

```
function saveUser(name: string, age: number) {  
  if (name == null || name == "") {  
    throw new Error("Name is Invalid");  
  }  
  
  if (age < 15 || age > 95) {  
    throw new Error("Age is Invalid");  
  }  
  
  this.userRepository.save(new User(name, age));  
}
```

```
function validateUser(name: string, age: number): boolean {  
  let isValid: boolean = true;  
  
  if (name == null || name == "") {  
    isValid = false;  
    throw new Error("Name is Invalid");  
  }  
  
  if (age < 15 || age > 95) {  
    isValid = false;  
    throw new Error("Age is Invalid");  
  }  
  
  return isValid;  
}
```

Query

```
function createUser(name: string, age: number) {  
  this.userRepository.save(new User(name, age));  
}
```

Command

# HOW DO YOU MAKE FUNCTIONS LIKE THIS?

---



**Revise Revise Revise!!!**

Write your code first and start chopping away at it until  
you have something that follows the rules mentioned

# EXCEPTION & ERROR HANDLING



Things can go wrong, we as programmers are responsible for making sure that our codes does what it needs to do => and should be clear!

# EXCEPTION & ERROR HANDLING

---



- **Returning an error code** creates two additional things that need to be done by the developer/client.
- Needs to know the mapping of each error code and at the same time he/she has to check the returning error code.
- For these cases, **throwing an exception** is better and will simplify the work of clients.
- Additional to this, the error handling will be 100% separated from your logic.



# ERROR HANDLING RULES



## ➤ Use Exceptions Rather Than Return Codes!

```
int withdraw(int amount) {  
    if (amount > _balance) {  
        return -1;  
    }  
    else {  
        balance -= amount;  
        return 0;  
    }  
}
```

*this method returns  
a special value that  
indicates an error*

**Throw an exception instead.**

```
void withdraw(int amount) throws BalanceException {  
    if (amount > _balance) {  
        throw new BalanceException();  
    }  
    balance -= amount;  
}
```

# ERROR HANDLING RULES



## ➤ Write Your Try-Catch-Finally Statement First

```
try {  
    // body of try  
}  
catch(error) {  
    // body of catch  
}
```

```
try {  
    // try_statements  
}  
catch(error) {  
    // catch_statements  
}  
finally() {  
    // codes that gets executed anyway  
}
```

✓ The main code is inside the `try` block. While executing the `try` block, if any error occurs, it goes to the `catch` block. The `catch` block handles the errors as per the catch statements.

✓ If no error occurs, the code inside the `try` block is executed and the `catch` block is skipped

- ✓ Your catch has to leave program in a consistent state, no matter what happens in the try.
- ✓ For this reason, it is good practice to start with a `try-catch-finally` statement when you are writing code that could throw exceptions.
- ✓ The `finally` block executes both when the code runs successfully or if an error occurs.

# ERROR HANDLING RULES

---



## ➤ Provide Context with Exceptions

- ✓ Each exception that you throw should provide enough context to determine the source and location of an error.
- ✓ Create informative error messages and pass them along with your exceptions.
- ✓ Mention the operation that failed and the type of failure.
- ✓ If you are logging in your application, pass along enough information to be able to log the error in your catch..

# ERROR HANDLING RULES



## ➤ Don't Return Null

- ✓ If you are tempted to return null from a method, consider throwing an exception
- ✓ If you are calling a null-returning method from a third-party API, consider wrapping that method with a method that either throws an exception

## ➤ Don't Pass Null

- ✓ Returning null from methods is bad but passing null into methods is worse.
- ✓ Unless you are working with an API which expects you to pass null, you should avoid passing null in your code whenever possible.

# FAQ: EXCEPTIONS AND ERROR HANDLING

---



<https://isocpp.org/wiki/faq/exceptions>

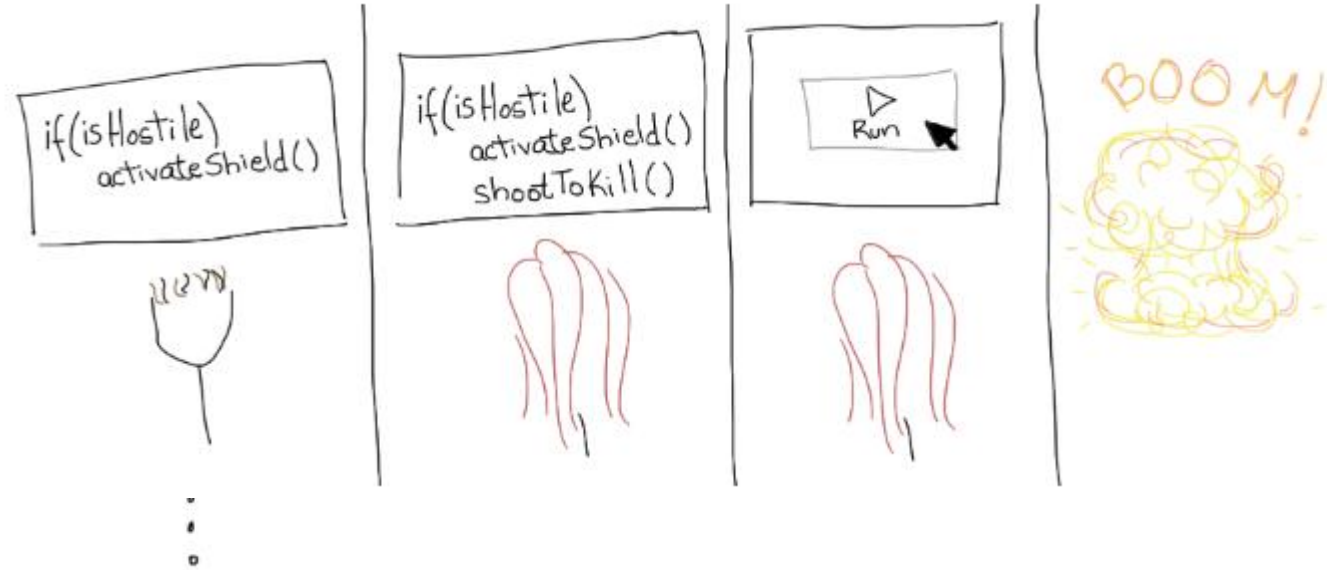
*Important contents with good explanation.*

*You must read it as you may get question from here in your coming exams....*

# TECHNIQUES THAT EVERY CODER SHOULD KNOW



- Keep It Simple
- Understand your Code
- Don't Repeat yourself
- Indent your code
- Naming convention
- Avoid Hard-Coded Strings and Magic Numbers
- Always Write the Braces Around Single Line Blocks
- Good Comments are your best friends
- Test run
- Write Everything Code-Related in English
- Practice your art

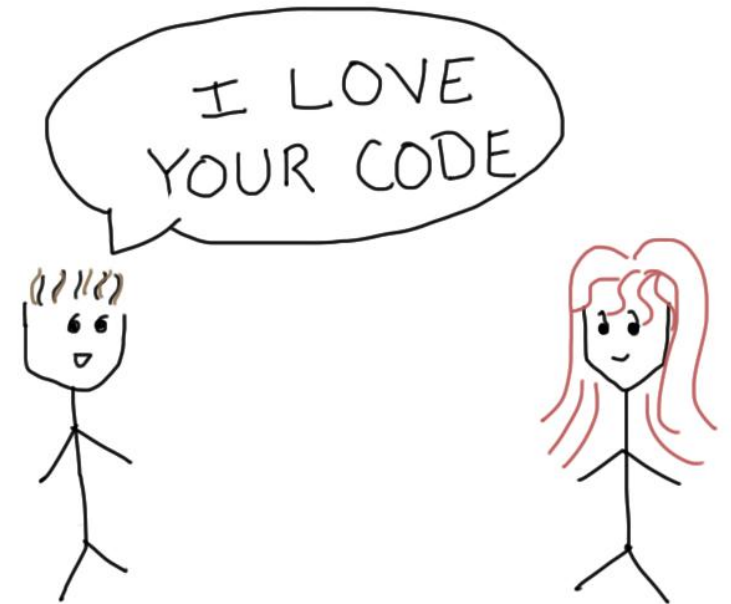


# MAKE PEOPLE HAPPY WITH YOUR CODE



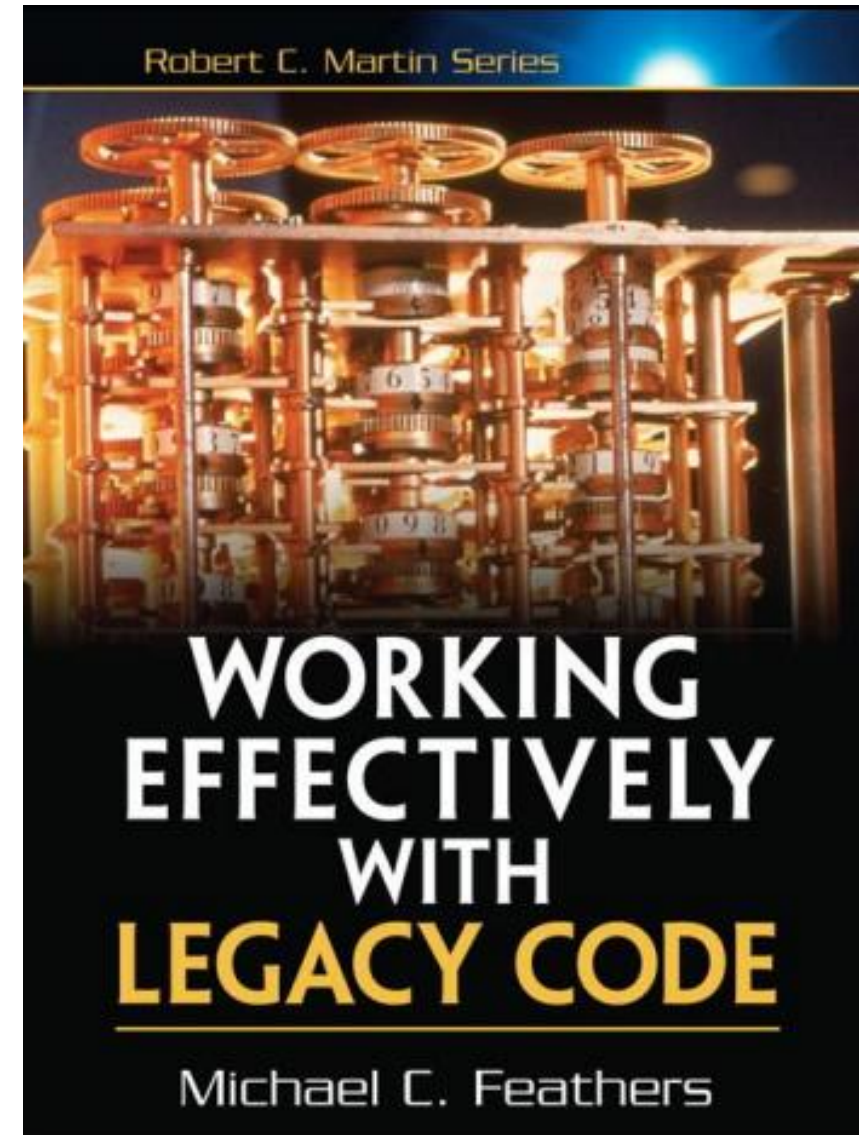
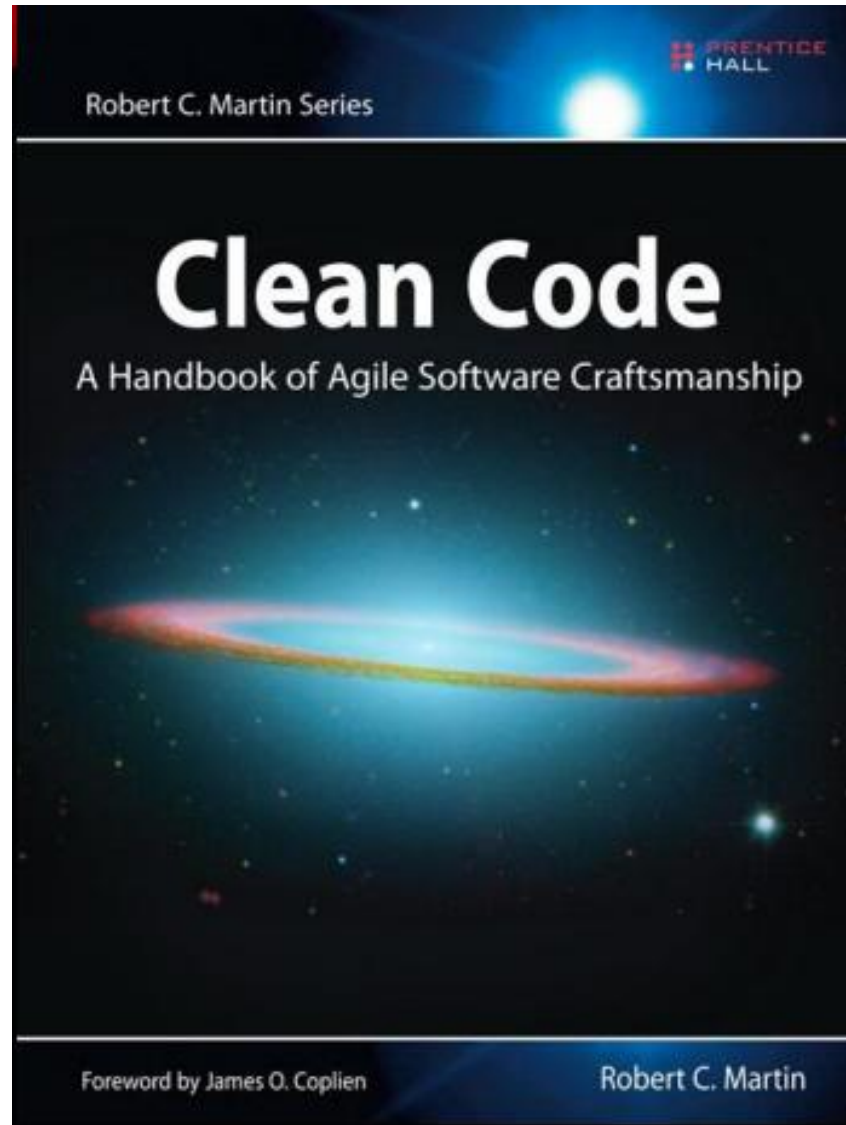
Through your code, you have the power to affect the life of your fellow programmers:

- ✓ They will rejoice/panic when they know they have to work with the code you wrote.
- ✓ They will spend 10 minutes/1 hour understanding the code you wrote.
- ✓ They will introduce 0 bugs/2 bugs when they edit the code you wrote.





# USEFUL BOOKS





# NEXT CLASS

---



NOT DECIDED YET 😊